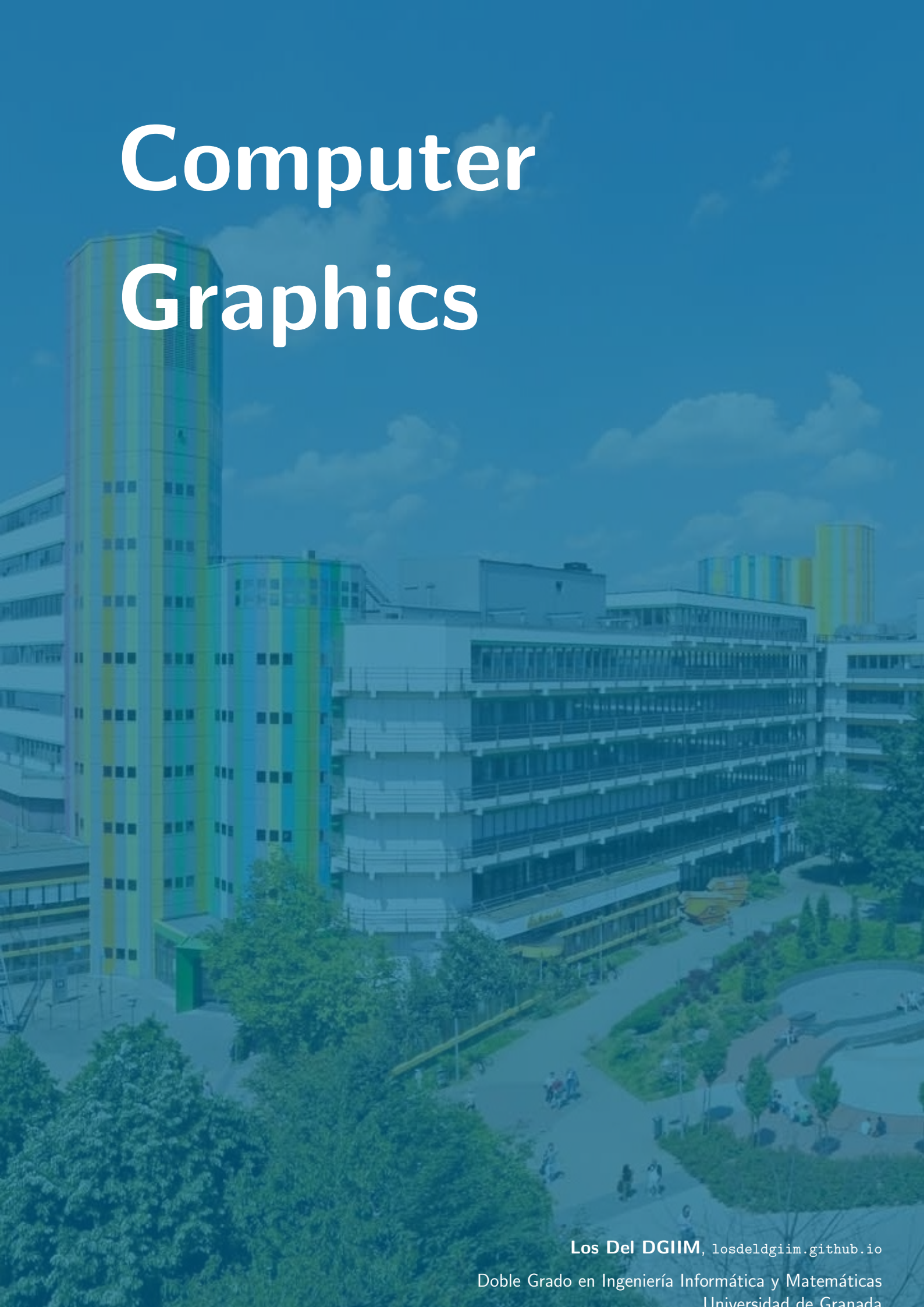
Computer Graphics



Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas

Universidad de Granada

Computer Graphics

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2026

Índice general

1. Ray and Path Tracing **5**
- 1.1. Ray Tracing . . . 6
  - 1.1.1. Diffuse surfaces . . . 7
- 1.2. Path Tracing . . . 7

2. Acceleration Structures **9**
- 2.1. Fewer rays . . . 9
- 2.2. Faster ray-object intersections . . . 9
  - 2.2.1. Barycentric interpolation . . . 9
  - 2.2.2. Ray-triangle intersection . . . 10
- 2.3. Fewer ray-object intersections . . . 11
  - 2.3.1. Bounding Volumes . . . 11
  - 2.3.2. Spatial subdivision . . . 13

3. Rasterization **17**
- 3.1. Fixed-Function Pipeline . . . 17
  - 3.1.1. Geometry . . . 17
  - 3.1.2. Geometry Processing . . . 18
  - 3.1.3. Rasterization . . . 23
  - 3.1.4. Fragment Processing . . . 24
- 3.2. Programmable Pipeline . . . 27

4. Color **29**
- 4.1. Technical Color Models . . . 29
  - 4.1.1. RGB Color Model . . . 29
  - 4.1.2. CMYK Color Model . . . 29
- 4.2. Perceptual Color Models . . . 29
  - 4.2.1. HSV Color Model . . . 29
  - 4.2.2. HSL Color Model . . . 29
  - 4.2.3. YUV Color Model . . . 29

1. Ray and Path Tracing

In order to convert a 3D scene into a 2D image that can be displayed on a screen, there are two different approaches:

- Rasterization (Object-ordered rendering): It follows the logic: "Which pixels does this object cover?" It projects the geometry (usually triangles) onto the 2D image plane and fills the corresponding pixels. Its main advantage is its efficiency.

- Ray Tracing (Image-ordered rendering): It follows the logic: "What light reaches this pixel?" It shoots rays from the observer's eye through each pixel into the scene, calculating light bounces, reflections, and shadows. Its main advantage is its ability to produce highly realistic images, as it simulates the physical behavior of light more accurately.

During this chapter, we will cover the basics of ray tracing and path tracing, two important techniques in computer graphics for rendering images that follow the first approach. In order to introduce them, some previous concepts are required.

Definición 1.1 (Light Ray)**.** A *light ray* is a straight line along which light energy (photons) travels. It is defined by its origin and direction (which can also be given as a second point).

Definición 1.2 (Light Path)**.** A *light path* is a sequence of light rays that represent the journey of light from its source to the observer, including any interactions with objects in the scene (such as reflection, refraction, or absorption).

When a light ray interacts with an object, it can be absorved or its direction can be changed due to reflection or refraction. In order to render images, there are two main approaches:

- Going forward: rays are traced forward from the light source, so the entire scene would be perfectly illuminated. However, most of the rays will not reach the observer, so this method is inefficient.

- Going backward: rays are traced backward from the observer, so only the rays that reach the observer are considered. A ray is traced for determining the color of each pixel in the image.

Given the inefficiency of the forward approach, the backward approach is the one used in ray tracing and path tracing.

1.1. Ray Tracing

Ray tracing[1] is a rendering technique that simulates the way light interacts with objects in a scene to produce realistic images. For each pixel in the image, a *primary ray* is traced from the observer's eye through the pixel into the scene. Once the ray intersects with an object, three type of *secondary rays* can be generated:

- Shadow rays: For each light source, a shadow ray is traced from the intersection point to the light source to determine if the point is in shadow or not. If the shadow ray intersects another object before reaching the light source, the point is in shadow and does not receive direct illumination from that light source.

- Reflection rays: If the surface is reflective, a reflection ray is traced in the direction of the reflected ray, which is calculated based on the angle of incidence and the surface normal.

- Refraction rays: If the surface is translucid, a refraction ray is traced in the direction of the refracted ray, which is calculated based on Snell's law.

Therefore, for each intersection $N + 2$ rays are traced, where N is the number of light sources in the scene. The reflection and refraction rays are recursively considered as primary rays, so they can generate more secondary rays if they intersect with other objects. This process continues until once of the following conditions is met:

- A ray does not intersect with any object, in which case it contributes the background color to the pixel.

- A ray reaches a predefined maximum recursion depth, in which case it does not generate any more rays and contributes just its local illumination to the pixel.

The color of each pixel is determined by the color of its first intersection point. For each intersection point, the color is calculated as:

$$I = k_d I_{\text{direct}} + k_s I_{\text{reflection}} + k_t I_{\text{refraction}}$$

where $k_d, k_s, k_t \in \mathbb{R}_0^+$ such that $k_d + k_s + k_t = 1$ are three coefficients that determine the contribution of each type of ray to the final color, and I_{direct} is the color contribution from local illumination calculated using phong shading, $I_{\text{reflection}}$ is the color contribution from the intersection point of the reflection ray, and $I_{\text{refraction}}$ is the color contribution from the intersection point of the refraction ray.

Observación. As the reader may have noticed, the complexity of ray tracing increases exponentially, but the contribution of rays decreases exponentially, so in practice, a maximum recursion depth of 4 or 5 is usually sufficient for producing high-quality images.

[1] It is also kown as Whitted Ray Tracing, as Turner Whitted was one of its original developpers.

1.1.1. Diffuse surfaces

There are two main types of surfaces:

- Diffuse surfaces: they reflect light uniformly in all directions.
- Specular surfaces: they reflect light in a specific direction, following the law of reflection.

As the reader may notice, ray tracing just considers specular surfaces. In order to render diffuse surfaces, another reason to stop the recursion is included:

- A ray hits a diffuse surface, in which case it does not generate any more rays and contributes just its local illumination to the pixel.

This way, between the eye and the light source as many specular surfaces as we want can be included, but only one diffuse surface can be included just before the light source.

1.2. Path Tracing

Path tracing is also a rendering technique that simulates the way light interacts with objects in a scene to produce realistic images, similar to ray tracing. For each pixel in the image, N rays are traced from the observer's eye through the pixel into the scene, where N is a predefined number of samples per pixel. Once a ray intersects with an object, its direction is randomly changed according to the material properties of the surface, and a new ray is traced in that direction, generating therefore a path. This process continues until one of the following conditions is met:

- A ray does not intersect with any object, in which case it contributes the background color to the pixel.
- A ray reaches a predefined maximum depth, in which case it contributes just its local illumination to the pixel.

The color of each pixel is determined by the average color contribution of all the rays traced for that pixel. It should however be noted that "randomly" does not mean "uniformly". Depending on the material properties of the surface, from an specific direction, some directions will be more likely to be chosen than others.

- For perfect diffuse surfaces, the new direction is chosen uniformly in the hemisphere above the surface.
- For perfect specular surfaces, the new direction is the reflection direction.

2. Acceleration Structures

One of the main problems of ray tracing is that for each ray, we need to calculate its first intersection point with the objects in the scene. This is $O(mp)$, where m is the number of objects in the scene and p is the number of pixels in the image (as many rays as pixels are traced). If there are n polygons in total in the scene, the complexity of ray tracing is $O(np)$. In order to reduce this complexity, acceleration structures are used. In the following sections, we will cover the three most common acceleration methods.

2.1. Fewer rays

Two main techniques can be used for reducing the number of rays traced:

- Adaptive tree-depth control: The maximum recursion depth is not fixed, but it is determined adaptively based on the contribution of each ray to the final image. If a ray contributes very little to the final image, it is not traced further.
- Stochastic sampling: As seen in path tracing, only one secondary ray is traced for each intersection point, whose direction is randomly chosen according to the material properties of the surface.

However, there are some cases where these techniques are not effective. In these cases, other acceleration methods are required.

2.2. Faster ray-object intersections

In order to speed up the ray-object intersection tests, the concept of "barycentric interpolation" in a triangle can be used.

2.2.1. Barycentric interpolation

Barycentric interpolation is a method of interpolating values at the vertices of a triangle to find the value at any point inside the triangle. It uses the concept of barycentric coordinates, which are a set of three numbers that represent the relative position of a point with respect to the vertices of the triangle. The barycentric coordinates of a point P with respect to a triangle defined by vertices P_1, P_2, and P_3 are given by:

$$\lambda_1 = \frac{\Delta(P, P_2, P_3)}{\Delta(P_1, P_2, P_3)}, \quad \lambda_2 = \frac{\Delta(P, P_3, P_1)}{\Delta(P_1, P_2, P_3)}, \quad \lambda_3 = \frac{\Delta(P, P_1, P_2)}{\Delta(P_1, P_2, P_3)}$$

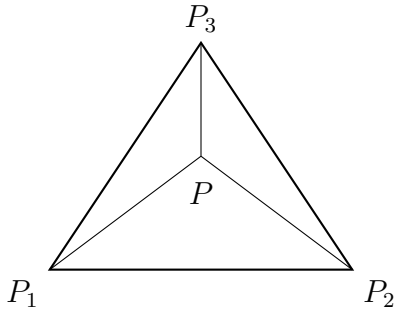


Figura 2.1: Barycentric coordinates of a point P with respect to a triangle defined by vertices P_1, P_2, and P_3.

where $\Delta(A, B, C)$ is the area of the triangle defined by points A, B, and C, as shown in Figure 2.1. If the point P is inside the triangle, then they satisfy the following properties:

- $\lambda_1 + \lambda_2 + \lambda_3 = 1$
- $0 \leqslant \lambda_i \leqslant 1$ for $i = 1, 2, 3$.

Once the barycentric coordinates of a point P are calculated, we can use them to interpolate any value defined at the vertices of the triangle. Given the values f_1, f_2, and f_3 at the vertices P_1, P_2, and P_3 respectively, the interpolated value at point P with barycentric coordinates λ_1, λ_2, and λ_3 is given by:

$$f(P) = \lambda_1 f_1 + \lambda_2 f_2 + \lambda_3 f_3$$

Two things should be noted about barycentric interpolation:

- It also interpolates the x and y coordinates of the point P in the triangle. These two conditions and the fact that $\lambda_1 + \lambda_2 + \lambda_3 = 1$ are sufficient to determine the barycentric coordinates of any point in the triangle.
- It can be proven that this interpolation is linear; and given that there is a unique linear interpolation for any set of values at the vertices of the triangle, it can be concluded that barycentric interpolation is just another way of performing linear interpolation.

2.2.2. Ray-triangle intersection

Any point P in the planed defined by a triangle defined by vertices P_1, P_2, and P_3 can be expressed as a linear combination of the vertices:

$$P = \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3$$

where λ_1, λ_2, and λ_3 are the barycentric coordinates of P with respect to the triangle. In addition, any point P' on a ray defined by an origin P_S and a direction $\overrightarrow{D}$ can be expressed as:

$$P' = P_S + t\overrightarrow{D} \qquad t \in \mathbb{R}_0^+$$

Therefore, determining the intersection point of a ray with a triangle is equivalent to finding the values of λ_1, λ_2, λ_3, and t that satisfy the following system of equations:

$$\begin{cases} \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3 = P_S + t\overrightarrow{D} \\ \lambda_1 + \lambda_2 + \lambda_3 = 1 \end{cases}$$

It should be noted that the first equation represents three equations, one for each coordinate. Therefore, we have a system of four equations with four unknowns. That system can be solved using Cramer's rule, but calculating the determinants can be computationally expensive. Instead, we can use the triple product of vectors to calculate the determinants more efficiently. After solving this system, we have to check if the solution is valid:

- $t \in \mathbb{R}_0^+$, as we are only interested in the intersection points that are in the direction of the ray (not behind the ray origin).
- $0 \leqslant \lambda_i \leqslant 1$ for $i = 1, 2, 3$, as we are only interested in the intersection points that are inside the triangle (not in the whole plane defined by the triangle).

This process is really fast, as:

- In order to determine λ_2, only one triple product of vectors is required. If λ_2 is not in the range $[0, 1]$, we can conclude that there is no intersection point between the ray and the triangle, and we can skip calculating λ_1 and λ_3.
- In order to determine λ_3, only a second triple product of vectors is required. If λ_3 is not in the range $[0, 1]$, we can conclude that there is no intersection point between the ray and the triangle.
- t is not even needed to be calculated, as the intersection point can be calculated using the barycentric coordinates.

2.3. Fewer ray-object intersections

There are two main techniques for reducing the number of ray-object intersection tests, which are developped in the next two sections.

2.3.1. Bounding Volumes

The main idea of bounding volumes is to enclose each object in the scene with a simple geometric shape (called a bounding volume) that can be tested for intersection with rays much faster than the original object (which may have thousands of polygons). There are two options:

- If the ray does not intersect the bounding volume, then it cannot intersect the original object, and we can skip the intersection test with the original object.
- If the ray intersects the bounding volume, then we need to perform the intersection test with the original object.

There are different types of bounding volumes, such as:

- Bounding spheres: The object is enclosed in a sphere defined by a center point and a radius.
  - Its intersection test with a ray is very fast, as it only requires solving a quadratic equation.
  - Its construction is also very fast, as it only requires calculating the center and the radius of the sphere that encloses the object. In addition, it doesn't have to be updated when the object is rigidly transformed.

  Despite these advantages, they also enclose a lot of empty space, which can lead to many false positives (i.e., rays that intersect the bounding sphere but do not intersect the original object).
- Axis-aligned bounding boxes (AABBs): The object is enclosed in a box whose edges are aligned with the coordinate axes.
  - Its intersection test with a ray is also fast.
  - Its construction is also very fast, as it only requires calculating the minimum and maximum coordinates of the object along each axis. However, it has to be updated when the object is rigidly transformed, as the minimum and maximum coordinates may change.
- Oriented bounding boxes (OBBs): The object is enclosed in a box that can be oriented in any direction.
  - Its intersection test with a ray is as fast as the AABB, but it can be more accurate, as it can fit the object better than the AABB.
  - Its construction is more expensive than the AABB, as it requires calculating the orientation of the box that best fits the object. However, it doesn't have to be reconstructed when the object is rigidly transformed, as the same transformation can be applied to the box.

However, using bounding volumes alone only reduces the number of ray-object intersection tests by a constant factor, as each object is still tested for intersection with the ray. The complexity of ray tracing with bounding volumes is still $O(np)$, but with a smaller constant factor. In order to achieve a better complexity, a hierarchy of bounding volumes can be used.

Bounding Volume Hierarchies (BVH)

A bounding volume hierarchy (BVH) is a tree structure where each node contains a bounding volume that encloses a subset of the objects contained in its father. The algorithm is therefore as follows:

- If the ray does not intersect the bounding volume of a node, then it cannot intersect any of the objects contained in that node, and we can skip the intersection tests with all those objects.

- If the ray intersects the bounding volume of a node, then we need to check the intersection with the bounding volumes of its children nodes, and so on until we reach the leaf nodes.

That way, we can reduce the number of ray-object intersection tests significantly, as many rays will not intersect the bounding volumes of the nodes in the hierarchy. The complexity of ray tracing with a BVH is significantly reduced to logarithmic.

2.3.2. Spatial subdivision

In this technique, instead of enclosing the objects in bounding volumes, the space is divided into disjoint regions (called cells), and each cell contains a list of the objects that are intersecting it. There are different types of spatial subdivision, such as the following.

Uniform grids

In this technique, the space is divided into a regular grid of cells, all of them of the same size. Traversing the grid is very fast, but it is a poor choice if the objects in the scene are not uniformly distributed, as many cells will be empty and others will contain many objects, leading to many ray-object intersection tests. Therefore, the following techniques are used to adapt the grid to the distribution of the objects in the scene.

Octrees

In this technique, the space is recursively subdivided into eight octants (hence the name octree). If one of the octants contains more than a certain number of objects, it is further subdivided into eight smaller octants, and so on until a certain maximum depth is reached or until all the octants contain a small number of objects. The result is a tree structure where each node represents an octant of the space, and each leaf node contains a octant that contains a small number of objects. This way, the grid is adapted to the distribution of the objects in the scene, leading to fewer ray-object intersection tests.

Each division is in fact three divisions:

- A division along the x-axis, with a plane perpendicular to the x-axis located at the middle of the cell along the x-axis, which divides the space in two equal parts along the x-axis.
- A division along the y-axis, with a plane perpendicular to the y-axis located at the middle of the cell along the y-axis, which divides the space in two equal parts along the y-axis.
- A division along the z-axis, with a plane perpendicular to the z-axis located at the middle of the cell along the z-axis, which divides the space in two equal parts along the z-axis.

KD trees

This idea follows the same idea as octrees, but instead of dividing the space into eight octants, each division uses just one plane.

- The plane direction is chosen to be perpendicular to one of the coordinate axes (i.e., x, y, or z). The order of the axes is chosen in a round-robin manner (i.e., x, then y, then z, then x again, and so on).
- There are however infinite possible positions for the plane along the chosen axis. In order to reduce the number of possible positions, only the planes located at the boundaries of the objects in the scene are considered. An optimization approach is used, selecting the plane that minimizes an specific cost function.

Binary Space Partitioning (BSP) trees

This idea follows the same idea as KD trees, but instead of restricting the plane direction to be perpendicular to one of the coordinate axes, any plane can be used for the division. Therefore, not only the direction but also the position of the plane can be optimized.

Detecting where a point P is located within the BSP tree is done by traversing the tree from the root node to the leaf nodes, at each node checking on which side of the plane the point P is located, and then moving to the corresponding child node until a leaf node is reached. In order to know on which side of the plane defined by a point P_0 and a normal vector $\overrightarrow{N}$ the point P is located, the following dot product can be calculated:

$$(P - P_0) \cdot \overrightarrow{N}$$

If the result is positive, then P is located on the side of the plane towards which the normal vector $\overrightarrow{N}$ is pointing. If the result is negative, then P is located on the opposite side of the plane. If the result is zero, then P is located on the plane itself.

Cost Function for KD and BSP trees

The cost function is used to determine the optimal position and direction of the dividing plane in both KD and BSP trees. An heuristic called *Surface Area Heuristic (SAH)* is commonly used for this purpose. It takes into account three components:

- The traversal cost, t_t, which is the cost of traversing the structure of the tree. It is usually constant, but it should be taken into account.
- The cost of processing each of the two parts of the space after the division. For each part, the cost is calculated taking three factors into account:
  - The probability of a ray intersecting that part of the space. As it depends on the surface area of that part of the space, it can be calculated as the ratio of the surface area of that part to the surface area of the whole space.

$$P_i = \frac{S_i}{S}$$

- The number of primitives in that part of the space, N_i. If one of the parts contains a lot of primitives, it should have a smaller probability of being intersected by a ray, as it will lead to more ray-object intersection tests.
- The cost of intersecting a ray with the primitives in that part of the space, t_p.

Therefore, the cost of processing each part of the space can be calculated as:

$$C_i = P_i \cdot N_i \cdot t_p$$

Therefore, the total cost of a division can be calculated as:

$$C = t_t + C_1 + C_2 = t_t + P_1 \cdot N_1 \cdot t_p + P_2 \cdot N_2 \cdot t_p$$

3. Rasterization

As we introduced in the first chapter, there are two main approaches for rendering images in computer graphics: rasterization and ray tracing. During this chapter, we will discuss the rasterization graphics pipeline. It starts from the initial creation of 3D models and ends with the final display of the rendered image. There are two types of graphics pipelines: the fixed-function pipeline and the programmable pipeline.

3.1. Fixed-Function Pipeline

The fixed-function pipeline is a traditional graphics pipeline that was widely used in older graphics hardware. It consists of a series of predefined stages that perform specific tasks. Each stage has a fixed function, and the data flows through these stages in a specific order. During the following subsections, we will discuss the stages of the fixed-function pipeline in detail.

3.1.1. Geometry

During this stage, the 3D models are created and defined. This includes defining the vertices, normals, texture coordinates, and other attributes of the 3D objects. In order to create the 3D models, the following primitives are used:

- `GL_POINTS`
- `GL_LINES`: A series of disconnected straight lines.
- `GL_LINE_STRIP`: A series of connected lines.
- `GL_LINE_LOOP`: A series of connected lines that form a loop.
- `GL_POLYGON`: A filled polygon defined by a series of vertices.
- `GL_TRIANGLES`: A series of disconnected triangles.
- `GL_TRIANGLE_STRIP`: A series of connected triangles. The first three vertices define the first triangle, and each subsequent vertex forms a new triangle with the previous two vertices.
- `GL_TRIANGLE_FAN`: A series of connected triangles that share a common central vertex. The first vertex is the center of the fan, and each subsequent vertex forms a new triangle with the center vertex and the previous vertex.
- `GL_QUADS`: A series of disconnected quadrilaterals.

- `GL_QUAD_STRIP`: A series of connected quadrilaterals. The first four vertices define the first quadrilateral, and each subsequent pair of vertices forms a new quadrilateral with the previous two vertices.

Even though all of these primitives are supported by OpenGL, all of the primitives will be converted to triangles, as they are the most optimized one in modern graphics cards. In order to specify the vertices of the primitives, a process evolved from immediate mode to vertex arrays and eventually to vertex buffer objects (VBOs).

1. Immediate Mode: In this mode, vertices are specified one at a time. It is simple to use but inefficient, as it requires multiple function calls for each vertex. The geometry is then stored in the code.

2. Vertex Arrays: This mode allows you to specify an array of vertices and their attributes, which can be more efficient than immediate mode. The geometry is then stored in the RAM.

3. Vertex Buffer Objects (VBOs): This mode allows you to store vertex data in the GPU's memory, which can significantly improve performance.

3.1.2. Geometry Processing

During this stage, the vertices defined in the geometry stage are processed to prepare them for rendering. There are different spaces in which the vertices are processed:

- Object Space: The original space in which the vertices are defined.

- World Space: The space in which the vertices are transformed to position them in the scene.

- Camera/Eye Space: The space in which the vertices are transformed to position them relative to the camera (the camera is at the origin).

- Clip Space: The space in which the vertices are transformed to prepare them for clipping (removing parts of the geometry that are outside the view frustum).

- Normalized Device Space: The space in which the vertices are transformed to fit within a standardized coordinate system (usually a cube from -1 to 1 in all axes).

- Screen Space: The final space in which the vertices are transformed to fit the actual screen dimensions.

In order to transform the vertices from one space to another, several transformations are applied.

Model Transformation

The model transformation is the transformation that takes the vertices from object space to world space. It is used to position the objects in the scene. It is usually represented as a combination of different transformations, and mathematically it can be represented as a single matrix M_{model} that is the product of the individual transformation matrices (taking the order of transformations into account).

$$P_{\text{world}} = M_{\text{model}} \cdot P_{\text{object}}$$

When a transformation M is applied to a vertex P, the normal vector N of the vertex is also transformed. Someone might think that the normal vector can be transformed using the same transformation M, but this is not always the case. The normal vector should be transformed using the inverse transpose of the transformation matrix M to ensure that it remains perpendicular to the surface after the transformation. Mathematically, the transformed normal vector N' can be calculated as follows:

$$N' = (M^{-1})^t \cdot N$$

Demostración. To understand why the normal vector should be transformed using the inverse transpose of the transformation matrix, we need to consider how transformations affect the geometry of the surface. Let n be the normal vector of a surface at a point P and t be the tangent vector at the same point. For clarity, let's use T_O to denote the transformation applied to the points (the tangent) and T_N to denote the transformation applied to the normal vector. We want to ensure that after applying the transformations, the normal vector remains perpendicular to the tangent vector, which means:

$$(T_N \cdot n) \perp (T_O \cdot t) \iff (T_N \cdot n) \cdot (T_O \cdot t) = 0 \iff (T_N \cdot n)^t \cdot (T_O \cdot t) = 0 \iff$$
$$\iff n^t \cdot T_N^t \cdot T_O \cdot t = 0$$

Given that n is perpendicular to t, we have $n^t \cdot t = 0$. Therefore, for the transformed normal vector to remain perpendicular to the transformed tangent vector, we need:

$$T_N^t \cdot T_O = I \iff T_N^t = T_O^{-1} \iff T_N = (T_O^{-1})^t$$

□

This should always be taken into account when applying transforming normals, *the normal vector should be transformed using the inverse transpose of the transformation matrix applied to the points.*

View Transformation

The view transformation is the transformation that takes the vertices from world space to camera/eye space. It is used to position the camera in the scene and to define the view direction. In OpenGL, the camera is always positioned at the origin of the coordinate system, looking in the z direction. As the camera can't move, the view transformation is achieved by applying the inverse of the camera's transformation to the vertices. Let's say that we want to position the camera at a point C in world space, looking at a direction D, with the up vector U'. We can define the view transformation using the following steps:

1. First of all, as we want to position the camera at point C, we need to apply a translation transformation that moves the world in the opposite direction of C. This can be represented using a translation matrix T that translates by $-C$.

2. Next, we need to align the camera's view direction with the z axis. To do this, we only know the direction D that the camera is looking at.

   a) We can calculate the right vector R of the camera using the cross product of the view direction D and the up vector U'. This can be represented as $R = D \times U'$.

   b) Then, we can calculate the up vector U of the camera using the cross product of the right vector R and the view direction D. This can be represented as $U = R \times D$. We need to do this step because the original up vector U' may not be perpendicular to the view direction D, so we need to calculate a new up vector U that is perpendicular to both R and D.

   This way, the matrix $[R, U, D]$ maps this orthogonal basis to the standard basis of the camera space. Therefore, the inverse of this transformation, which aligns the camera space with the standard basis, is:

$$[R, U, D]^{-1}$$

Therefore, the view transformation can be represented as the product of the translation and the inverse of the rotation:

$$M_{\text{view}} = [R, U, D]^{-1} \cdot T$$

The order of the transformations is important, as the translation should be applied before the rotation to ensure that the camera is correctly positioned and oriented in the scene.

Lighting and Shading

Before moving to the next stage, lighting and shading should be considered, because they are the processes that determine the color and intensity of the pixels in the final rendered image. Lighting is the process of calculating how light interacts with the surfaces of the objects in the scene, while shading is the process of determining the color of each pixel based on the lighting calculations. The Phong Lighting Model is a widely used model for calculating the lighting in computer graphics. It uses the following notation.

Notación. The notation used in the Phong Lighting Model is shwon in Figure 3.1.

- X is the point on the surface being shaded.
- N is the normal unitary vector at point X.
- L is the unitary vector from point X to the light source.

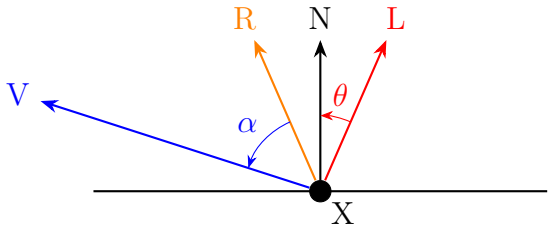


Figura 3.1: Notation used in the Phong Lighting Model.

- V is the unitary vector from point X to the viewer (camera).
- R is the reflection of the light vector L around the normal vector N, calculated as $R = 2(N \cdot L)N - L$.

  This calculation is valid because $N \cdot L = \cos(\theta)$ gives the projection of L onto N (as the cosine is the adjacent side of the triangle formed by L and N, and $\|L\| = 1$).

The Phong Lighting Model calculates the color of a pixel using three components: ambient, diffuse, and specular.

- Ambient Component: This component represents the constant background light in the scene. It is calculated as:

  $$\text{Ambient}(X) = C_{X,a} \cdot C_{L,a}$$

  where $C_{X,a}$ is the ambient color of the surface at point X and $C_{L,a}$ is the ambient color of the light source.

- Diffuse Component: This component represents the light that is scattered in all directions when it hits a surface. It is calculated as:

  $$\text{Diffuse}(L, X) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, \cos(\theta)) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, N \cdot L)$$

  where $C_{X,d}$ is the diffuse color of the surface at point X, $C_{L,d}$ is the diffuse color of the light source, and θ is the angle between the normal vector N and the light vector L.

- Specular Component: This component represents the light that is reflected in a specific direction when it hits a surface. It is calculated as:

  $$\text{Specular}(L, X, V) = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, \cos(\alpha))^n = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, R \cdot V)^n$$

  where $C_{X,s}$ is the specular color of the surface at point X, $C_{L,s}$ is the specular color of the light source, α is the angle between the reflection vector R and the view vector V, and n is the shininess coefficient that controls the size of the specular highlight.

The final color of the pixel is then calculated as the sum of these three components:

$$\text{Color}(L, X, V) = \text{Ambient}(X) + \text{Diffuse}(L, X) + \text{Specular}(L, X, V)$$

Perspective Transform

In this stage, the vertices are transformed from camera/eye space to clip space. The next definition is important.

Definición 3.1 (Frustrum)**.** The view frustum is a geometric shape that represents the volume of space that is visible to the camera. It can be defined in several ways (using the field of view, just the planes...). Two important planes are:

- Near Plane: The plane that is closest to the camera. Objects that are closer than this plane will not be rendered.
- Far Plane: The plane that is farthest from the camera. Objects that are farther than this plane will not be rendered.

Two type of frustrums can be defined:

- Perspective Frustum: A frustum that represents a perspective projection, where objects that are farther from the camera appear smaller. This type of frustum looks like a truncated pyramid, with the near plane being smaller than the far plane.
- Orthographic Frustum: A frustum that represents an orthographic projection, where objects that are farther from the camera appear the same size as objects that are closer to the camera. This type of frustum looks like a rectangular box, with the near and far planes being the same size. This type of projection is often used for 2D rendering or for technical drawings where accurate measurements are important. It represents that the viewer is at the infinity.

As already explained, the perspective transformation is used to transform the vertices from camera/eye space to clip space. It is represented as a matrix that prepares the vertices for perspective division. The exact form of the perspective transformation matrix depends on the parameters of the view frustum (how was it given), but it generally includes terms that account for the field of view, aspect ratio, and the near and far planes. We will simply call it $M_{\text{projection}}$, without going into the details of its construction. Therefore, what the programmer should implement in OpenGL is:

$$P_{\text{clip}} = M_{\text{projection}} \cdot M_{\text{view}} \cdot M_{\text{model}} \cdot P_{\text{object}}$$

Clipping

The clipping stage removes any vertices that are outside the view frustum. Here, the vertices are transformed from clip space to normalized device space by performing perspective division. Given that the vertices are in homogeneous coordinates and we need the last w coordinate to be 1, the other coordinates are divided by w to achieve this. After perspective division, and given that the correct perspective transformation was applied, the vertices will be in normalized device space, and only the vertices that are within the range of $[-1, 1]$ in all axes will be kept for further processing. The vertices that are outside this range will be clipped, meaning that they will be discarded and not rendered.

Viewport Transformation

In this last stage, the vertices are transformed from normalized device space to screen space. This transformation maps the normalized device coordinates to the actual pixel coordinates on the screen. The viewport transformation is defined by the viewport parameters, which specify the position and size of the viewport on the screen. The viewport transformation can be represented as a matrix that scales and translates the vertices to fit within the specified viewport. After this transformation, the vertices are ready to be rasterized and rendered on the screen.

3.1.3. Rasterization

The rasterization stage is the process of converting the vertices and primitives into pixels on the screen. During this stage, the graphics pipeline determines which pixels on the screen correspond to each primitive, and a fragment is generated for each of these pixels. The data for each vertex (color, texture coordinates, etc.) is interpolated across the primitive to determine the attributes of each fragment.

Data Interpolation

Given $n+1$ points $\overrightarrow{x}_i \in \mathbb{R}^k$ with their corresponding values $f_i \in \mathbb{R}$, $i \in \{0, \ldots, n\}$, interpolating is finding a function $f : \mathbb{R}^k \to \mathbb{R}$ such that $f(\overrightarrow{x}_i) = f_i$ for all $i \in \{0, \ldots, n\}$. When achieved, the function f can be used to estimate the value of f at any point $\overrightarrow{x} \in \mathbb{R}^k$ by evaluating $f(\overrightarrow{x})$.

Linear interpolation is a simple method of interpolation that only considers the linear functions f, which can be represented as $f(\overrightarrow{x}) = \overrightarrow{a} \cdot \overrightarrow{x} + b$ for some $\overrightarrow{a} \in \mathbb{R}^k$ and $b \in \mathbb{R}$. Given that we have $n + 1$ constraints (one for each point) and $k + 1$ unknowns (the components of $\overrightarrow{a}$ and b), we can solve for the coefficients of the linear function f if $n + 1 \leqslant k + 1$. In our case, we are interested in interpolating the attributes of the vertices across the primitives, which means that we have 3 vertices (for triangles) and two dimensions (the screen coordinates), so $k = 2$. Therefore, our aim is to find $a, b, c \in \mathbb{R}$ such that $f(x, y) = ax + by + c$ and $f(\overrightarrow{x}_i) = f_i$ for $i \in \{0, 1, 2\}$. This can be achieved by solving the system of equations of Equation (3.1).

$$\begin{cases} ax_0 + by_0 + c = f_0 \\ ax_1 + by_1 + c = f_1 \\ ax_2 + by_2 + c = f_2 \end{cases} \implies \begin{pmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} \tag{3.1}$$

However, this does not take the perspective into account. To achieve perspective-correct interpolation, the depth value of each vertex is interpolated in a way that accounts for the perspective distortion; and then the attributes are interpolated using the interpolated depth values.

Shading

Shading is the process of determining the color of each pixel based on the lighting calculations. During rasterization, the attributes of the vertices are interpolated across the primitive to determine the attributes of each fragment. There are different

ways to apply the Phong Lighting Mode, depending on what is interpolated and when is the color calculated:

- Flat Shading: The color is calculated for each primitive (e.g., triangle) using a single value of each attribute for the whole primitive (usually the corresponding values of the first vertex). This results in a faceted appearance, as each primitive has a single color.
- Gouraud Shading: The color is calculated at each vertex , and then interpolated across the primitive. This results in a smoother appearance than flat shading, but it can produce artifacts such as Mach bands.
- Phong Shading: The normal vector is interpolated across the primitive, and the color is calculated for each fragment using the interpolated normal vector. This results in a smoother appearance than Gouraud shading and can produce more accurate lighting effects.

3.1.4. Fragment Processing

The fragment processing stage is almost the final stage of the graphics pipeline, where the fragments generated during rasterization are processed to determine their final color and whether they should be rendered on the screen. During this stage, several operations can be performed on the fragments, such as depth testing, stencil testing, blending, and more. The specific operations performed during fragment processing depend on the settings and configurations of the graphics pipeline.

Textures

A texture map is a discretely sampled multi-dimensional function containing material properties, like color or normals. Texture mapping is the process of applying a texture map to a 3D model to add detail and realism to the rendered image. During fragment processing, the texture coordinates of each fragment are used to sample the corresponding attribute from the texture map, which can then be used. Some concepts about textures are:

- Wrapping: Texture coordinates are usually planned to be in the range of $[0, 1]$, but sometimes they can be outside this range. Wrapping is the process of determining how to handle texture coordinates that are outside the $[0, 1]$ range. Common wrapping modes include:
  - `GL_REPEAT`: The texture is repeated when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_MIRRORED_REPEAT`: The texture is repeated and mirrored when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_CLAMP_TO_EDGE`: The texture coordinates are clamped to the edge of the texture when they are outside the $[0, 1]$ range. This means that the texture will be stretched to fill the area outside the $[0, 1]$ range.

  One mode should be selected for each texture coordinate (e.g., s and t).

- Filtering: Usually, the texture maps are of a different resolution than the rendered image, which means that the texture needs to be filtered to determine the color of each pixel. There are two types of filtering:
  - Minification: The process of determining the color of a fragment when the texels (texture pixels) are smaller than the screen pixels.
  - Magnification: The process of determining the color of a fragment when the texels are larger than the screen pixels.

  Common filtering modes include:
  - `GL_NEAREST`: The color of the fragment is determined by the color of the nearest texel. In minification, this can lead to aliasing artifacts, while in magnification, it can lead to a blocky appearance.
  - `GL_LINEAR`: The color of the fragment is determined by the bilinear interpolation of the colors of the 4 nearest texels. In minification it does not make a big difference (considering 4 texels when the pixel may be covered by 100 texels does not improve the quality), but in magnification it can improve the appearance of the texture, making it appear smoother.
- MIP Mapping: MIP mapping is a technique used to improve the quality of textures when they are minificated. It involves creating multiple levels of detail for a texture, where each level is a downscaled version of the original texture. During rendering, the appropriate level of detail is selected based on the distance and angle of the fragment being rendered. This can help to reduce aliasing artifacts and improve the overall appearance of the texture, as no minification is applied when the appropriate level of detail is selected.

Fragment Tests

After the fragment processing stage, several tests can be performed on the fragments to determine whether they should be rendered on the screen or discarded. Before performing these tests, the content of each pixel should be explained. Each pixel and fragment on the screen has a buffer that stores:

- Color: The color of the pixel, usually represented as RGB (red, green, blue).
- Alpha: The alpha value of the pixel, which people tend to use it to represent the transparency of the pixel. However, it can be used for other purposes as well.
- Depth: The depth value of the current pixel, which is the depth value of the fragment that is currently rendered at that pixel (the closest fragment to the camera). It is stored in the depth buffer.
- Stencil: The stencil value of the pixel, which is used for stencil testing. The programmer can define the stencil value for each pixel and store it in the stencil buffer.

Once this has been explained, some common fragment tests include:

- Alpha Test: This test discards fragments based on their alpha value. It consists of a comparison between the alpha value of the *fragment* and a *reference value*, using a specified comparison function (e.g., less than, greater than, equal to). If the comparison fails, the fragment is discarded and not rendered on the screen.
- Depth Test: This test discards fragments based on their depth value. It compares the depth value of the *fragment* with the depth value of the corresponding *pixel* in the depth buffer. Different comparison functions can be used (e.g., less than, greater than, equal to) to determine whether the fragment should be discarded or rendered. However, the most common comparison function is "less than", (`GL_LESS`), which means that a fragment will only be rendered if it is closer to the camera than the existing pixel. If that is the case, it passes the depth test and is rendered on the screen, and the depth buffer is updated with the new depth value. If the fragment is farther from the camera than the existing pixel, it fails the depth test and is discarded.
- Stencil Test: This test discards fragments based on their stencil value. It compares the stencil value of the *pixel* where the fragment would be rendered with a *reference value*, using a specified comparison function. If the comparison fails, the fragment is discarded and not rendered on the screen. The stencil test can be used for various effects, such as creating shadows, outlining objects, or implementing complex masking operations.

Blending

Blending is the process of combining the color of a fragment with the color of the corresponding pixel in the framebuffer to produce a final color that is rendered on the screen. Blending is typically (but not only) used to achieve transparency effects, where the color of the fragment is blended with the color of the background to create a semi-transparent appearance. The blending operation is defined by:

$$C_{\text{final}} = C_{\text{source}} \cdot F_{\text{source}} \odot C_{\text{destination}} \cdot F_{\text{destination}}$$

where:

- C_{final} is the final color that is rendered on the screen.
- C_{source} is the color of the fragment being processed.
- $C_{\text{destination}}$ is the color of the corresponding pixel in the framebuffer.
- F_{source} and $F_{\text{destination}}$ are the blending factors for the source and destination colors, respectively. These factors determine how much of each color contributes to the final color. Common blending factors include:
  - `GL_ZERO`: The factor is 0, meaning that the corresponding color does not contribute to the final color.
  - `GL_ONE`: The factor is 1, meaning that the corresponding color fully contributes to the final color.

  - `GL_SRC_ALPHA`: The factor is equal to the alpha value of the source color, meaning that the contribution of the source color is proportional to its transparency.
  - `GL_ONE_MINUS_SRC_ALPHA`: The factor is equal to 1 minus the alpha value of the source color, meaning that the contribution of the destination color is proportional to the transparency of the source color.

  The specific blending factors used can be configured based on the desired effect.

- $\odot$ represents the operator used to combine the source and destination colors. Common operators include addition, subtraction, minimum, and maximum.

The well knwon α-blending is achieved by using the following blending factors and operator:

- F_{source} = `GL_SRC_ALPHA`
- $F_{\text{destination}}$ = `GL_ONE_MINUS_SRC_ALPHA`
- $\odot$ is the addition operator.

This is a common blending mode used to achieve transparency effects. However, there are many other blending modes that can be used to achieve different effects. For instance, the following blending mode can be used to count how many fragments are rendered at each pixel, which can be useful for various effects such as outlining objects or creating shadows:

- F_{source} = `GL_ONE`
- $F_{\text{destination}}$ = `GL_ONE`
- $\odot$ is the addition operator.

3.2. Programmable Pipeline

The programmable pipeline is a modern graphics pipeline that allows developers to write custom shaders to control the behavior of each stage of the pipeline. This provides greater flexibility and control over the rendering process, allowing for more complex and realistic graphics. The programmable pipeline consists of several stages, including vertex shading, tessellation, geometry shading, fragment shading, and more. Each stage can be programmed using a shading language such as GLSL (OpenGL Shading Language) or HLSL (High-Level Shading Language). The programmable pipeline has largely replaced the fixed-function pipeline in modern graphics hardware, as it allows for more advanced rendering techniques and greater performance.

4. Color

4.1. Technical Color Models

4.1.1. RGB Color Model

4.1.2. CMYK Color Model

4.2. Perceptual Color Models

4.2.1. HSV Color Model

4.2.2. HSL Color Model

4.2.3. YUV Color Model